



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

ОТЧЕТ

по Лабораторной работе № 4

по дисциплине «**МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ**»

Тема: «**Методы поиска условного экстремума**»

Студент	<u>ИУСИ-22М</u>	<u></u>	<u>Тавуз Мохамад</u>
	(группа)	(подпись, дата)	(И.О. Фамилия)
Преподаватель		<u></u>	<u>Ю.Е.Гапанюк</u>
		(подпись, дата)	(И.О. Фамилия)

2026 г.

1) Введение

Методы условной оптимизации играют важную роль в задачах математического программирования и машинного обучения, так как во многих практических задачах необходимо минимизировать целевую функцию при наличии ограничений на переменные. В отличие от задач безусловной оптимизации, в данном случае решение должно не только обеспечивать минимальное значение функции, но и принадлежать области допустимых решений.

В данной лабораторной работе рассматривается задача поиска условного минимума тестовой функции Розенброка при заданных ограничениях. Цель работы — изучение и программная реализация алгоритмов условной оптимизации, а также сравнение их эффективности по точности, количеству итераций, времени выполнения и выполнению ограничений.

В ходе работы были реализованы следующие методы: метод внешних штрафных функций, метод внутренних барьерных функций, комбинированный метод штрафных и барьерных функций, метод модифицированных функций Лагранжа и метод проекции градиента. Для каждого метода был найден приближенный условный минимум, выполнена проверка допустимости решения и построены графики сходимости.

2) Описание задания

В соответствии с заданием лабораторной работы необходимо найти условный минимум тестовой функции Розенброка с учетом ограничений. В работе рассматривается вариант 2, для которого заданы следующие параметры: $a = 150$, $b = 2$, $f_0 = 100$, $n = 3$.

Тестовая функция имеет вид:

$$f(x) = \sum_{i=1}^{n-1} a(x_{i+1} - x_i^2)^2 + b(x_i - 1)^2 + f_0.$$

Для выбранного варианта при $n = 3$ функция принимает вид:

$$f(x_1, x_2, x_3) = 150(x_2 - x_1^2)^2 + 2(x_1 - 1)^2 + 150(x_3 - x_2^2)^2 + 2(x_2 - 1)^2 + 100.$$

Ограничения для варианта 2 имеют следующий вид:

$$g_1(x) = x_1^2 + x_2^2 + x_3^2 - 1 \leq 0,$$

$$g_2(x) = -x_1 \leq 0,$$

$$g_3(x) = -x_2 \leq 0,$$

$$g_4(x) = -x_3 \leq 0.$$

Таким образом, область допустимых решений представляет собой часть единичной сферы в неотрицательном октанте, то есть $x_1 \geq 0$, $x_2 \geq 0$, $x_3 \geq 0$ и $x_1^2 + x_2^2 + x_3^2 \leq 1$.

В ходе выполнения лабораторной работы необходимо реализовать следующие методы условной оптимизации:

- метод внешних штрафных функций;
- метод внутренних барьерных функций;
- комбинированный метод штрафных и барьерных функций;
- метод модифицированных функций Лагранжа;
- метод проекции градиента.

Также необходимо сравнить полученные решения по значению целевой функции, количеству итераций, времени выполнения, норме градиента и величине нарушения ограничений.

3) Текст программы и экранные формы с примерами выполнения

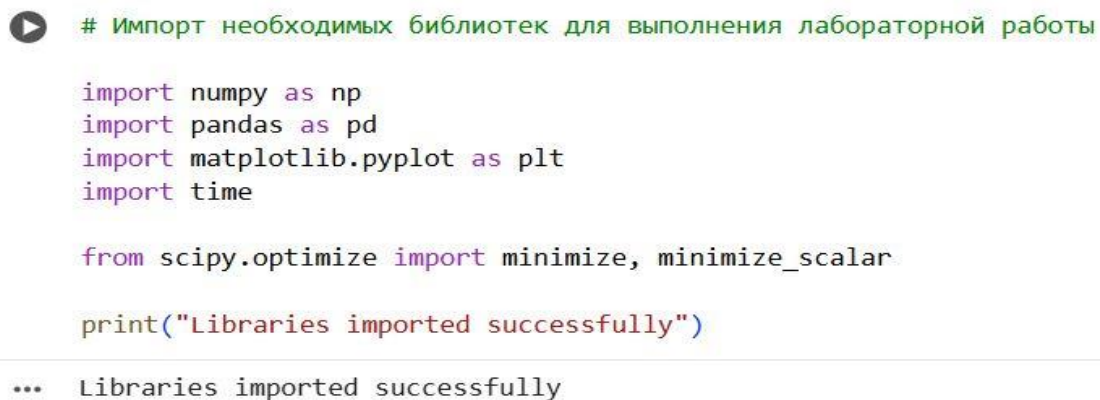
В данном разделе приведены основные этапы программной реализации методов условной оптимизации. Все вычисления были выполнены в среде Google Colab с использованием языка Python.

Для численных вычислений использовалась библиотека `numpy`, для формирования таблиц результатов — `pandas`, для построения графиков — `matplotlib`. Также была использована функция `minimize` из модуля `scipy.optimize`, применяемая для решения вспомогательных задач безусловной минимизации, возникающих в методах штрафных функций, барьерных функций и модифицированных функций Лагранжа.

3.1. Импорт необходимых библиотек

На первом этапе были подключены основные библиотеки, необходимые для выполнения лабораторной работы. Для работы с векторами и численными вычислениями использовалась библиотека `numpy`. Для формирования сравнительных таблиц результатов была подключена библиотека `pandas`. Для построения графиков сходимости применялась библиотека `matplotlib`. Также была подключена функция `minimize` из библиотеки `scipy.optimize`, которая использовалась для решения вспомогательных задач безусловной минимизации внутри реализованных методов условной оптимизации.

После подключения библиотек была выполнена проверка корректности импорта. В результате программа вывела сообщение об успешной загрузке библиотек.



```
# Импорт необходимых библиотек для выполнения лабораторной работы

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from scipy.optimize import minimize, minimize_scalar

print("Libraries imported successfully")

... Libraries imported successfully
```

Рисунок 1 — Импорт необходимых библиотек

3.2. Задание параметров варианта и определение функции Розенброка

На следующем этапе были заданы параметры варианта 2: $a = 150$, $b = 2$, $f_0 = 100$, $n = 3$. После этого была определена тестовая функция Розенброка, используемая для проверки работы алгоритмов условной оптимизации.

Так как $n = 3$, функция зависит от трех переменных x_1 , x_2 и x_3 . Для функции без учета ограничений теоретический минимум достигается в точке $x^* = (1, 1, 1)$. При подстановке этой точки значение функции равно $f(x^*) = 100$. Однако в данной лабораторной работе рассматривается задача условной оптимизации, поэтому необходимо дополнительно учитывать выполнение ограничений.

```
▶ # Задание параметров варианта и определение тестовой функции Розенброка

variant = 2

a = 150
b = 2
f0 = 100
n = 3

def rosenbrock_function(x):
    x = np.array(x, dtype=float)
    value = 0.0
    for i in range(n - 1):
        value += a * (x[i + 1] - x[i] ** 2) ** 2 + b * (x[i] - 1) ** 2
    return value + f0

x_star_unconstrained = np.ones(n)
f_star_unconstrained = rosenbrock_function(x_star_unconstrained)

print("Variant:", variant)
print("a =", a)
print("b =", b)
print("f0 =", f0)
print("n =", n)
print("Unconstrained theoretical minimum point:", x_star_unconstrained)
print("Function value at unconstrained theoretical minimum:", f_star_unconstrained)

... Variant: 2
a = 150
b = 2
f0 = 100
n = 3
Unconstrained theoretical minimum point: [1. 1. 1.]
Function value at unconstrained theoretical minimum: 100.0
```

Рисунок 2 — Задание параметров варианта и функции Розенброка

3.3. Определение функций ограничений и проверка допустимости точек

После определения целевой функции были заданы функции ограничений для выбранного варианта. Первое ограничение задает принадлежность точки единичной сфере, а остальные ограничения обеспечивают неотрицательность всех координат.

Для проверки корректности реализации ограничений были рассмотрены несколько тестовых точек. Точка (1, 1, 1) не является допустимой, так как нарушает первое ограничение $x_1^2 + x_2^2 + x_3^2 - 1 \leq 0$. Точки (0.5, 0.5, 0.5) и (0.2, 0.3, 0.4) удовлетворяют всем ограничениям, поэтому являются допустимыми.

Также была реализована функция вычисления нарушения ограничений. Она используется далее для оценки качества найденных решений и проверки их принадлежности допустимой области.

```
▶ # Определение функций ограничений для выбранного варианта

def constraints(x):
    x = np.array(x, dtype=float)
    g1 = x[0] ** 2 + x[1] ** 2 + x[2] ** 2 - 1
    g2 = -x[0]
    g3 = -x[1]
    g4 = -x[2]
    return np.array([g1, g2, g3, g4], dtype=float)

def constraint_violation(x):
    g = constraints(x)
    return np.sum(np.maximum(g, 0.0) ** 2)

def is_feasible(x, tolerance=1e-8):
    return np.all(constraints(x) <= tolerance)

test_points = [
    np.array([1.0, 1.0, 1.0]),
    np.array([0.5, 0.5, 0.5]),
    np.array([0.2, 0.3, 0.4])
]

for point in test_points:
    print("Point:", point)
    print("g(x):", constraints(point))
    print("Constraint violation:", constraint_violation(point))
    print("Feasible:", is_feasible(point))
    print("-" * 50)
```

Рисунок 3 — Код определения функций ограничений и проверки допустимости точек

```

... Point: [1. 1. 1.]
    g(x): [ 2. -1. -1. -1.]
    Constraint violation: 4.0
    Feasible: False
-----
    Point: [0.5 0.5 0.5]
    g(x): [-0.25 -0.5 -0.5 -0.5 ]
    Constraint violation: 0.0
    Feasible: True
-----
    Point: [0.2 0.3 0.4]
    g(x): [-0.71 -0.2 -0.3 -0.4 ]
    Constraint violation: 0.0
    Feasible: True
-----

```

Рисунок 4 — Результат проверки допустимости тестовых точек

3.4. Определение градиента функции Розенброка

Для работы реализуемых методов был определен градиент функции Розенброка. Градиент необходим для анализа направления изменения целевой функции и используется в методе проекции градиента, а также для оценки качества найденного решения.

Корректность вычисления градиента была проверена в теоретической точке минимума функции без учета ограничений $x^* = (1, 1, 1)$. В этой точке все компоненты градиента равны нулю, а норма градиента также равна 0. Это подтверждает, что точка x^* является стационарной точкой для безусловной задачи.

Однако проверка ограничений показала, что точка $(1, 1, 1)$ не принадлежит допустимой области, так как первое ограничение принимает положительное значение. Поэтому для задачи условной оптимизации необходимо искать другое решение, удовлетворяющее всем ограничениям.

```

▶ # Определение градиента функции Розенброка и проверка в теоретической точке

def rosenbrock_gradient(x):
    x = np.array(x, dtype=float)
    grad = np.zeros_like(x)

    for i in range(n - 1):
        grad[i] += -4 * a * x[i] * (x[i + 1] - x[i] ** 2) + 2 * b * (x[i] - 1)
        grad[i + 1] += 2 * a * (x[i + 1] - x[i] ** 2)

    return grad

gradient_at_unconstrained_minimum = rosenbrock_gradient(x_star_unconstrained)
gradient_norm_at_unconstrained_minimum = np.linalg.norm(gradient_at_unconstrained_minimum)

print("Gradient at unconstrained theoretical minimum:", gradient_at_unconstrained_minimum)
print("Gradient norm at unconstrained theoretical minimum:", gradient_norm_at_unconstrained_minimum)
print("Feasible unconstrained theoretical minimum:", is_feasible(x_star_unconstrained))
print("Constraints at unconstrained theoretical minimum:", constraints(x_star_unconstrained))

```

Рисунок 5 — Код определения градиента функции Розенброка

```

... Gradient at unconstrained theoretical minimum: [0. 0. 0.]
Gradient norm at unconstrained theoretical minimum: 0.0
Feasible unconstrained theoretical minimum: False
Constraints at unconstrained theoretical minimum: [ 2. -1. -1. -1.]

```

Рисунок 6 — Проверка градиента и допустимости теоретической точки

3.5. Задание начальной точки и параметров алгоритмов

Перед запуском алгоритмов были заданы общие параметры вычислений. В качестве начальной точки была выбрана точка $x_0 = (0.2, 0.2, 0.2)$. Данная точка находится внутри допустимой области, так как все ее координаты неотрицательны, а значение $x_1^2 + x_2^2 + x_3^2$ меньше единицы.

Также были заданы критерии остановки алгоритмов: точность по норме градиента $\epsilon_{\text{grad}} = 10^{-6}$, точность по изменению аргумента $\epsilon_{\text{x}} = 10^{-8}$, точность по изменению значения функции $\epsilon_{\text{f}} = 10^{-10}$ и максимальное количество итераций $\text{max_iter} = 1000$.

Проверка начальной точки показала, что все ограничения выполняются, а величина нарушения ограничений равна нулю. Это особенно важно для метода барьерных функций, так как он требует выбора начальной точки внутри допустимой области.

```
# Задание начальной точки и общих параметров вычислений

x0 = np.array([0.2, 0.2, 0.2], dtype=float)

epsilon = 1e-6
epsilon_x = 1e-8
epsilon_f = 1e-10
max_iter = 1000

print("Initial point:", x0)
print("Function value at initial point:", rosenbrock_function(x0))
print("Gradient at initial point:", rosenbrock_gradient(x0))
print("Gradient norm at initial point:", np.linalg.norm(rosenbrock_gradient(x0)))
print("Constraints at initial point:", constraints(x0))
print("Constraint violation at initial point:", constraint_violation(x0))
print("Feasible initial point:", is_feasible(x0))

print("epsilon =", epsilon)
print("epsilon_x =", epsilon_x)
print("epsilon_f =", epsilon_f)
print("max_iter =", max_iter)
```

Рисунок 7 — Код задания начальной точки и параметров вычислений

```
... Initial point: [0.2 0.2 0.2]
Function value at initial point: 110.24000000000001
Gradient at initial point: [-22.4  25.6  48. ]
Gradient norm at initial point: 58.83128419472075
Constraints at initial point: [-0.88 -0.2 -0.2 -0.2 ]
Constraint violation at initial point: 0.0
Feasible initial point: True
epsilon = 1e-06
epsilon_x = 1e-08
epsilon_f = 1e-10
max_iter = 1000
```

Рисунок 8 — Проверка начальной точки и параметров вычислений

3.6. Реализация метода внешних штрафных функций

На данном этапе был реализован метод внешних штрафных функций. Основная идея метода заключается в преобразовании исходной задачи условной оптимизации в последовательность задач безусловной минимизации. Для этого к целевой функции добавляется штрафное слагаемое, которое увеличивает значение вспомогательной функции при нарушении ограничений.

В реализации была использована штрафная функция, основанная на квадрате положительной части ограничений. Если ограничение выполняется, оно не вносит вклад в штраф. Если ограничение нарушается, значение штрафа возрастает пропорционально квадрату величины нарушения. На каждой внешней итерации выполнялась минимизация вспомогательной функции методом BFGS, после чего параметр штрафа увеличивался.

В результате метод внешних штрафных функций нашел допустимое приближенное решение. Максимальное нарушение ограничений оказалось меньше заданной точности, поэтому найденная точка может рассматриваться как решение задачи условной оптимизации.

```

# Реализация метода внешних штрафных функций

def penalty_function_value(x, r):
    g = constraints(x)
    penalty = np.sum(np.maximum(g, 0.0) ** 2)
    return rosenbrock_function(x) + r * penalty

def external_penalty_method(x_start, r0=1.0, c=10.0, eps=1e-6, max_outer_iter=30):
    start_time = time.time()

    x_current = np.array(x_start, dtype=float)
    r = r0
    history = []

    for k in range(max_outer_iter):
        result = minimize(
            lambda x: penalty_function_value(x, r),
            x_current,
            method="BFGS",
            options={
                "gtol": 1e-8,
                "maxiter": 1000,
                "disp": False
            }
        )

        x_current = result.x
        f_current = rosenbrock_function(x_current)
        violation_current = max_constraint_violation(x_current)

        history.append({
            "iteration": k + 1,
            "r": r,
            "x": x_current.copy(),
            "f_value": f_current,
            "constraint_violation": violation_current,
            "penalty_value": penalty_function_value(x_current, r)
        })

        if violation_current <= eps:
            break

        r *= c

    execution_time = time.time() - start_time

    return create_result(
        method_name="External penalty method",
        x=x_current,
        iterations=len(history),
        history=history,
        execution_time=execution_time
    )

penalty_result = external_penalty_method(x0)

print_result(penalty_result)
print("Constraints at found point:", constraints(penalty_result["x"]))

```

Рисунок 9 — Код реализации метода внешних штрафных функций

```

... Method: External penalty method
Found minimum point: [0.7548918  0.57044138 0.3236285 ]
Function value: 100.48972020809242
Iterations: 7
Gradient norm: 1.6450756124289698
Max constraint violation: 4.127105741602577e-07
Feasible: True
Execution time: 0.15660715103149414
Constraints at found point: [ 4.12710574e-07 -7.54891804e-01 -5.70441381e-01 -3.23628502e-01]

```

Рисунок 10 — Результат работы метода внешних штрафных функций

3.7. Реализация метода внутренних барьерных функций

Далее был реализован метод внутренних барьерных функций. В отличие от метода внешних штрафов, данный метод строит последовательность точек внутри допустимой области и не позволяет алгоритму выходить за ее границы.

В работе была использована логарифмическая барьерная функция. Если текущая точка находится внутри допустимой области, то для каждого ограничения вычисляется слагаемое $-\ln(-g_i(x))$. При приближении к границе допустимой области значение барьерной функции возрастает, что препятствует выходу точки за пределы области допустимых решений.

Начальная точка $x_0 = (0.2, 0.2, 0.2)$ была выбрана внутри допустимой области, поэтому она подходит для запуска метода барьерных функций. На каждой итерации выполнялась минимизация вспомогательной функции методом BFGS, а параметр барьера постепенно уменьшался. В результате метод нашел допустимое решение, удовлетворяющее всем ограничениям.

```
# Реализация метода внутренних барьерных функций

def barrier_function_value(x, r):
    g = constraints(x)

    if np.any(g >= 0):
        return 1e20

    barrier = -np.sum(np.log(-g))
    return rosenbrock_function(x) + r * barrier

def barrier_method(x_start, r0=1.0, c=10.0, eps=1e-6, max_outer_iter=30):
    start_time = time.time()

    x_current = np.array(x_start, dtype=float)
    r = r0
    history = []

    for k in range(max_outer_iter):
        result = minimize(
            lambda x: barrier_function_value(x, r),
            x_current,
            method="BFGS",
            options={
                "gtol": 1e-8,
                "maxiter": 1000,
                "disp": False
            }
        )

        x_current = result.x
        f_current = rosenbrock_function(x_current)
        violation_current = max_constraint_violation(x_current)

        history.append(f
```

```

        history.append({
            "iteration": k + 1,
            "r": r,
            "x": x_current.copy(),
            "f_value": f_current,
            "constraint_violation": violation_current,
            "barrier_value": barrier_function_value(x_current, r)
        })

    if r <= eps:
        break

    r /= c

execution_time = time.time() - start_time

return create_result(
    method_name="Barrier method",
    x=x_current,
    iterations=len(history),
    history=history,
    execution_time=execution_time
)

barrier_result = barrier_method(x0)

print_result(barrier_result)
print("Constraints at found point:", constraints(barrier_result["x"]))

```

Рисунок 11 — Код реализации метода внутренних барьерных функций

```

... Method: Barrier method
Found minimum point: [0.75486473 0.57039778 0.32358006]
Function value: 100.4898205166785
Iterations: 8
Gradient norm: 1.6449387231902644
Max constraint violation: 0.0
Feasible: True
Execution time: 0.0858147144317627
Constraints at found point: [-1.21564891e-04 -7.54864725e-01 -5.70397777e-01 -3.23580063e-01]

```

Рисунок 12 — Результат работы метода внутренних барьерных функций

3.8. Реализация комбинированного метода штрафных и барьерных функций

На следующем этапе был реализован комбинированный метод штрафных и барьерных функций. Данный подход объединяет идеи внешнего штрафа и внутреннего барьера. Штрафное слагаемое используется для учета возможного нарушения ограничений, а барьерное слагаемое препятствует выходу точки за границы допустимой области.

В реализации целевая функция была дополнена двумя компонентами: квадратичным штрафом за нарушение ограничений и логарифмическим барьером для допустимых точек. Если точка выходила за пределы допустимой области, барьерная часть принимала большое значение, что делало такие точки невыгодными для алгоритма.

На каждой итерации выполнялась минимизация комбинированной вспомогательной функции методом BFGS. Параметр штрафа постепенно увеличивался, а параметр барьера уменьшался. В результате метод нашел допустимое решение, близкое к решениям, полученным другими методами условной оптимизации.



Реализация комбинированного метода штрафных и барьерных функций

```
def combined_function_value(x, r_penalty, r_barrier):
    g = constraints(x)

    penalty = np.sum(np.maximum(g, 0.0) ** 2)

    if np.any(g >= 0):
        barrier = 1e10
    else:
        barrier = -np.sum(np.log(-g))

    return rosenbrock_function(x) + r_penalty * penalty + r_barrier * barrier

def combined_penalty_barrier_method(
    x_start,
    r_penalty0=1.0,
    r_barrier0=1.0,
    c=10.0,
    eps=1e-6,
    max_outer_iter=30
):
    start_time = time.time()

    x_current = np.array(x_start, dtype=float)
    r_penalty = r_penalty0
    r_barrier = r_barrier0
    history = []

    for k in range(max_outer_iter):
```

```

for k in range(max_outer_iter):
    result = minimize(
        lambda x: combined_function_value(x, r_penalty, r_barrier),
        x_current,
        method="BFGS",
        options={
            "gtol": 1e-8,
            "maxiter": 1000,
            "disp": False
        }
    )

    x_current = result.x
    f_current = rosenbrock_function(x_current)
    violation_current = max_constraint_violation(x_current)

    history.append({
        "iteration": k + 1,
        "r_penalty": r_penalty,
        "r_barrier": r_barrier,
        "x": x_current.copy(),
        "f_value": f_current,
        "constraint_violation": violation_current,
        "combined_value": combined_function_value(x_current, r_penalty, r_barrier)
    })

    if violation_current <= eps and r_barrier <= eps:
        break

    r_penalty *= c
    r_barrier /= c

execution_time = time.time() - start_time

return create_result(
    method_name="Combined penalty-barrier method",
    x=x_current,
    iterations=len(history),
    history=history,
    execution_time=execution_time
)

combined_result = combined_penalty_barrier_method(x0)

print_result(combined_result)
print("Constraints at found point:", constraints(combined_result["x"]))

```

Рисунок 13 — Код реализации комбинированного метода штрафных и барьерных функций

```
... Method: Combined penalty-barrier method
Found minimum point: [0.75488075 0.5704451 0.32364695]
Function value: 100.48972074162099
Iterations: 8
Gradient norm: 1.6500074372882227
Max constraint violation: 0.0
Feasible: True
Execution time: 0.19638967514038086
Constraints at found point: [-1.01067361e-07 -7.54880748e-01 -5.70445097e-01 -3.23646947e-01]
```

Рисунок 14 — Результат работы комбинированного метода штрафных и барьерных функций

3.9. Реализация метода модифицированных функций Лагранжа

Далее был реализован метод модифицированных функций Лагранжа. В отличие от обычного метода штрафных функций, в данном подходе штрафное слагаемое добавляется не только к целевой функции, а используется в составе модифицированной функции Лагранжа с учетом множителей Лагранжа.

В реализации для ограничений вида $g_i(x) \leq 0$ использовалась модифицированная функция Лагранжа с неотрицательными множителями μ_i . После решения очередной вспомогательной задачи безусловной минимизации выполнялось обновление множителей Лагранжа по правилу $\mu_i = \max(0, \mu_i + r g_i(x))$. Это позволяет учитывать активные ограничения и повышает устойчивость метода.

В результате метод модифицированных функций Лагранжа нашел допустимое решение за небольшое количество итераций. По сравнению с методами штрафных и барьерных функций данный метод показал более быструю сходимость по числу внешних итераций.

```

# Реализация метода модифицированных функций Лагранжа

def augmented_lagrangian_value(x, mu, r):
    g = constraints(x)
    terms = np.maximum(0.0, g + mu / r)
    return rosenbrock_function(x) + (r / 2.0) * np.sum(terms ** 2) - (1.0 / (2.0 * r)) * np.sum(mu ** 2)

def modified_lagrangian_method(x_start, r0=1.0, c=5.0, eps=1e-6, max_outer_iter=30):
    start_time = time.time()

    x_current = np.array(x_start, dtype=float)
    mu = np.zeros(4, dtype=float)
    r = r0
    history = []

    for k in range(max_outer_iter):
        result = minimize(
            lambda x: augmented_lagrangian_value(x, mu, r),
            x_current,
            method="BFGS",
            options={
                "gtol": 1e-8,
                "maxiter": 1000,
                "disp": False
            }
        )

        x_current = result.x
        g_current = constraints(x_current)
        f_current = rosenbrock_function(x_current)
        violation_current = max_constraint_violation(x_current)

        history.append({
            "iteration": k + 1,
            "r": r,
            "mu": mu.copy(),
            "x": x_current.copy(),
            "f_value": f_current,
            "constraint_violation": violation_current,
            "lagrangian_value": augmented_lagrangian_value(x_current, mu, r)
        })

        if violation_current <= eps:
            break

        mu = np.maximum(0.0, mu + r * g_current)
        r *= c

    execution_time = time.time() - start_time

    return create_result(
        method_name="Modified Lagrangian method",
        x=x_current,
        iterations=len(history),
        history=history,
        execution_time=execution_time
    )

modified_lagrangian_result = modified_lagrangian_method(x0)

print_result(modified_lagrangian_result)
print("Constraints at found point:", constraints(modified_lagrangian_result["x"]))

```

Рисунок 15 — Код реализации метода модифицированных функций Лагранжа

```
... Method: Modified Lagrangian method
Found minimum point: [0.75489308 0.57043991 0.32362759]
Function value: 100.48972048745384
Iterations: 5
Gradient norm: 1.6445704292340748
Max constraint violation: 7.074028629538986e-08
Feasible: True
Execution time: 0.05377197265625
Constraints at found point: [ 7.07402863e-08 -7.54893080e-01 -5.70439908e-01 -3.23627594e-01]
```

Рисунок 16 — Результат работы метода модифицированных функций Лагранжа

3.10. Реализация метода проекции градиента

Последним был реализован метод проекции градиента. Данный метод относится к прямым методам условной оптимизации, так как на каждой итерации строится новая точка, принадлежащая допустимой области. В отличие от методов штрафов и барьеров, здесь ограничения учитываются непосредственно через операцию проекции.

В реализации сначала выполнялся шаг в направлении антиградиента, после чего полученная точка проецировалась на допустимую область. Для данной задачи допустимая область задается условиями $x_i \geq 0$ и $x_1^2 + x_2^2 + x_3^2 \leq 1$. Поэтому проекция включала зануление отрицательных координат и нормировку вектора в случае выхода за пределы единичной сферы.

Для повышения устойчивости метода был использован адаптивный выбор шага. Это позволило избежать расходимости и получить допустимое решение, близкое к результатам остальных методов. В качестве критерия остановки использовалась норма проецированного градиента.

```

# Реализация метода проекции градиента с адаптивным выбором шага

def project_to_feasible_region(x):
    x = np.maximum(x, 0.0)
    norm_x = np.linalg.norm(x)

    if norm_x > 1.0:
        x = x / norm_x

    return x

def projected_gradient_norm(x):
    projected_point = project_to_feasible_region(x - rosenbrock_gradient(x))
    return np.linalg.norm(x - projected_point)

def gradient_projection_method_adaptive(
    x_start,
    alpha0=0.1,
    beta=0.5,
    eps=1e-6,
    max_iter=5000
):
    start_time = time.time()

    x = project_to_feasible_region(np.array(x_start, dtype=float))
    history = []

    for k in range(max_iter):
        grad = rosenbrock_gradient(x)
        pg_norm = projected_gradient_norm(x)

        history.append({
            "iteration": k + 1,
            "x": x.copy(),
            "f_value": rosenbrock_function(x),
            "gradient_norm": np.linalg.norm(grad),
            "projected_gradient_norm": pg_norm,
            "constraint_violation": max_constraint_violation(x)
        })

        if pg_norm < eps:
            break

        alpha = alpha0
        f_current = rosenbrock_function(x)

        while alpha > 1e-12:
            x_candidate = project_to_feasible_region(x - alpha * grad)

            if rosenbrock_function(x_candidate) <= f_current - 1e-4 * np.linalg.norm(x_candidate - x) ** 2:
                break

            alpha *= beta

        x_new = x_candidate

        if np.linalg.norm(x_new - x) < epsilon_x:
            x = x_new
            break

        x = x_new

    execution_time = time.time() - start_time

    return create_result(
        method_name="Gradient projection method",
        x=x,
        iterations=len(history),
        history=history,
        execution_time=execution_time
    )

projection_result = gradient_projection_method_adaptive(x0)

print_result(projection_result)
print("Projected gradient norm:", projected_gradient_norm(projection_result["x"]))
print("Constraints at found point:", constraints(projection_result["x"]))

```

Рисунок 17 — Код реализации метода проекции градиента с адаптивным выбором шага

```

... Method: Gradient projection method
Found minimum point: [0.75489307 0.57043988 0.32362757]
Function value: 100.48972054562253
Iterations: 199
Gradient norm: 1.6445698362877585
Max constraint violation: 0.0
Feasible: True
Execution time: 0.0346522331237793
Projected gradient norm: 1.8202522335482624e-06
Constraints at found point: [-1.11022302e-16 -7.54893066e-01 -5.70439880e-01 -3.23627567e-01]

```

Рисунок 18 — Результат работы метода проекции градиента с адаптивным выбором шага

3.11. Сравнение результатов работы методов

После реализации всех методов была сформирована сравнительная таблица результатов. В таблицу были включены найденные координаты точки минимума, значение целевой функции, количество итераций, норма градиента, максимальное нарушение ограничений, признак допустимости решения и время выполнения каждого метода.

Из полученных результатов видно, что все реализованные методы нашли допустимые решения, удовлетворяющие ограничениям задачи. Значения целевой функции для всех методов оказались близкими и находятся около $f(x) \approx 100.49$. Это показывает, что разные методы условной оптимизации сходятся к одной и той же области решения.

Наименьшее количество итераций показал метод модифицированных функций Лагранжа. Метод проекции градиента потребовал большего количества итераций, однако также нашел допустимое решение. Все методы дали значение нарушения ограничений, не превышающее заданную точность.

...	Method	x1	x2	x3	Function value	Iterations	Gradient norm	Max constraint violation	Feasible	Execution time
0	External penalty method	0.754892	0.570441	0.323629	100.489720	7	1.645076	4.127106e-07	True	0.156607
1	Barrier method	0.754865	0.570398	0.323580	100.489821	8	1.644939	0.000000e+00	True	0.085815
2	Combined penalty-barrier method	0.754881	0.570445	0.323647	100.489721	8	1.650007	0.000000e+00	True	0.196390
3	Modified Lagrangian method	0.754893	0.570440	0.323628	100.489720	5	1.644570	7.074029e-08	True	0.053772
4	Gradient projection method	0.754893	0.570440	0.323628	100.489721	199	1.644570	0.000000e+00	True	0.034652

Рисунок 19 — Сравнительная таблица результатов методов условной оптимизации

3.12. Анализ сходимости методов по значению целевой функции

Для анализа поведения реализованных алгоритмов был построен график изменения значения целевой функции по итерациям. Данный график позволяет оценить, насколько быстро каждый метод приближается к условному минимуму функции Розенброка.

По графику видно, что все методы уменьшают значение целевой функции и сходятся к близким значениям. Методы внешних штрафных функций, барьерных функций, комбинированный метод и метод модифицированных функций Лагранжа достигают результата за небольшое количество внешних итераций, так как на каждой внешней итерации решается вспомогательная задача безусловной минимизации.

Метод проекции градиента имеет более длинную траекторию сходимости, так как он выполняет непосредственное итерационное движение с проекцией каждой новой точки на допустимую область. Несмотря на большее количество итераций, метод также сходится к значению целевой функции, близкому к результатам остальных методов.

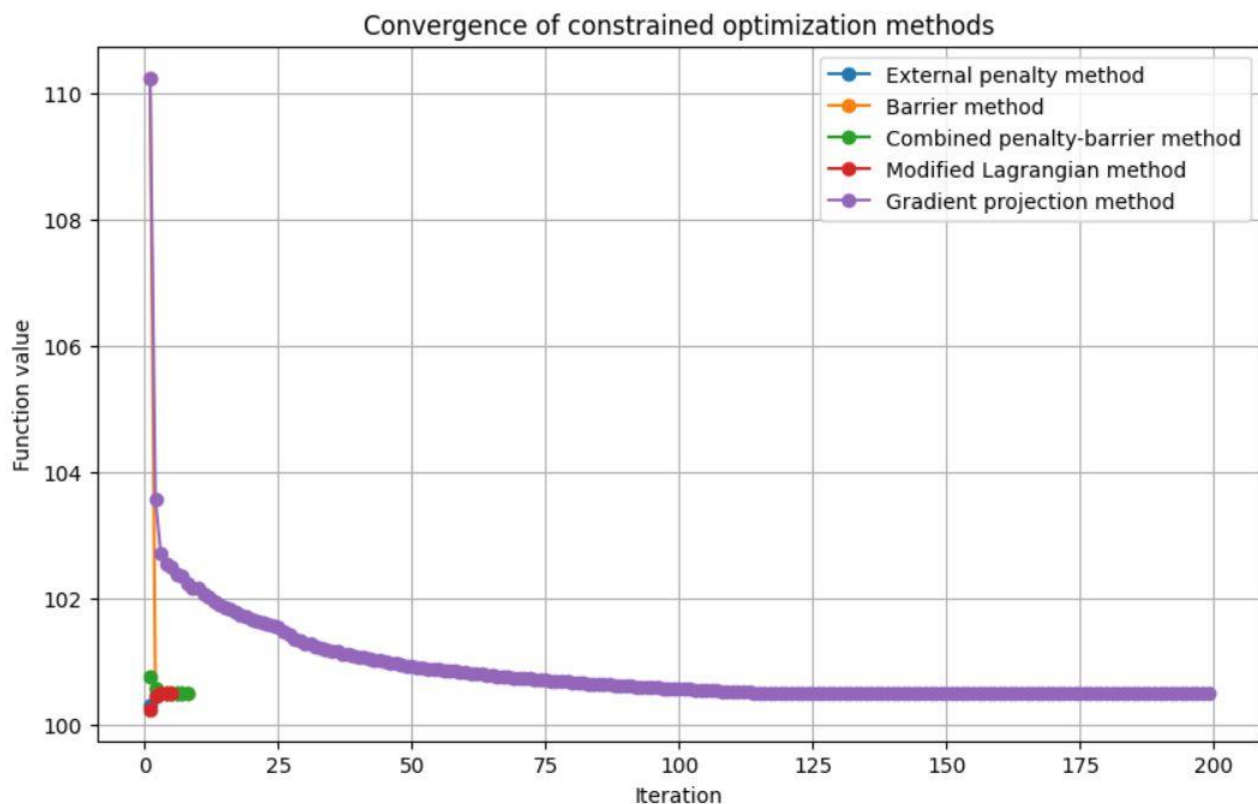


Рисунок 20 — График сходимости методов по значению целевой функции

3.13. Анализ изменения нарушения ограничений

Дополнительно был построен график изменения максимального нарушения ограничений по итерациям. Данный график позволяет оценить, насколько корректно каждый метод учитывает ограничения задачи в процессе поиска условного минимума.

Из графика видно, что все методы в конечном результате обеспечивают выполнение ограничений. Для методов, основанных на штрафных и барьерных функциях, нарушение ограничений постепенно уменьшается по мере изменения параметров вспомогательной функции. В методе внешних штрафов это связано с увеличением параметра штрафа, а в методе барьерных функций — с движением внутри допустимой области.

Метод проекции градиента сохраняет допустимость точек за счет операции проекции на допустимую область. Поэтому итоговое нарушение ограничений для данного метода равно нулю. Полученные результаты подтверждают, что реализованные алгоритмы корректно учитывают ограничения выбранного варианта.

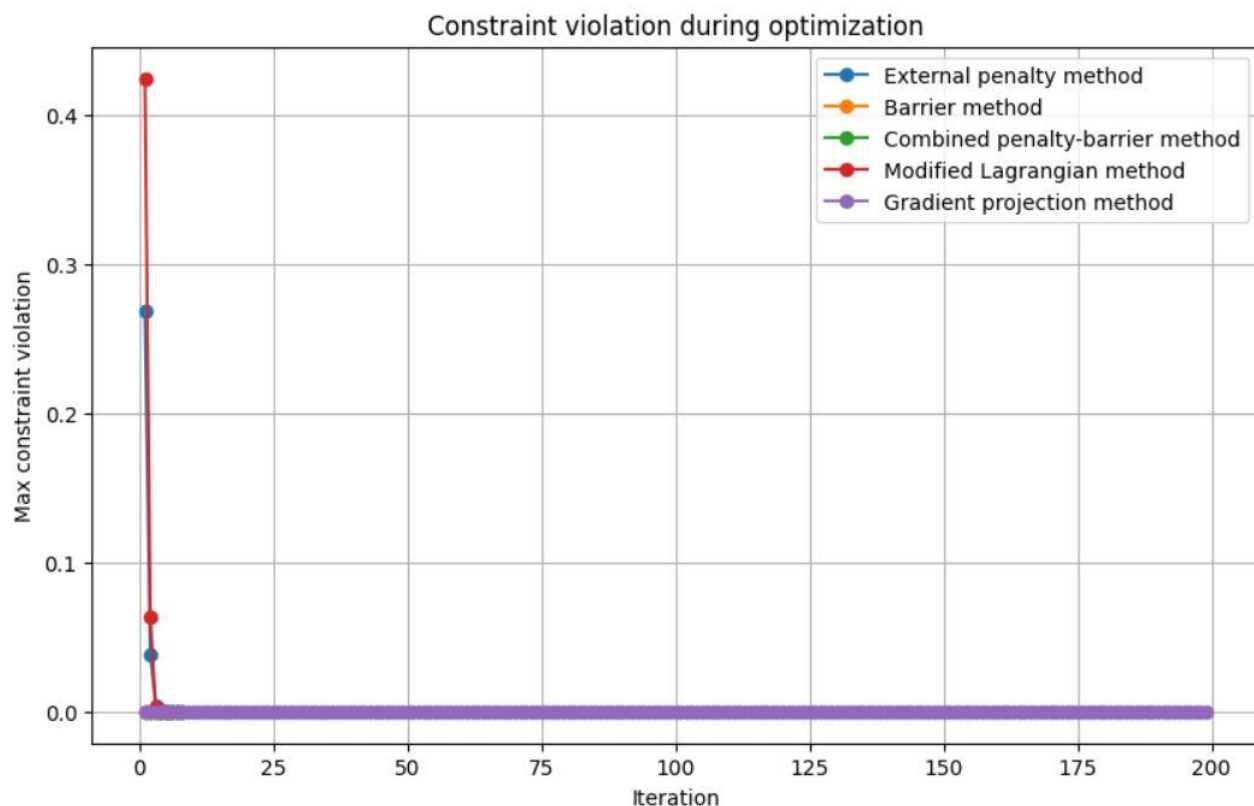


Рисунок 21 — График изменения нарушения ограничений по итерациям

3.14. Проверка найденных решений на допустимость

Одним из обязательных этапов решения задачи условной оптимизации является проверка найденных точек на допустимость. Для этого для каждого метода были вычислены значения всех функций ограничений $g_1(x)$, $g_2(x)$, $g_3(x)$, $g_4(x)$.

Результаты проверки показали, что для всех найденных решений выполняется условие $g_i(x) \leq 0$ с учетом заданной численной точности. Это означает, что полученные точки принадлежат допустимой области. Следовательно, найденные решения можно рассматривать как корректные приближения условного минимума функции Розенброка для выбранного варианта.

Особенно важно отметить, что теоретическая точка минимума безусловной задачи $x^* = (1, 1, 1)$ не является допустимой, поэтому полученное условное решение отличается от нее и находится на границе допустимой области.

...	Method	g1	g2	g3	g4	All constraints <= 0
0	External penalty method	4.127106e-07	-0.754892	-0.570441	-0.323629	True
1	Barrier method	-1.215649e-04	-0.754865	-0.570398	-0.323580	True
2	Combined penalty-barrier method	-1.010674e-07	-0.754881	-0.570445	-0.323647	True
3	Modified Lagrangian method	7.074029e-08	-0.754893	-0.570440	-0.323628	True
4	Gradient projection method	-1.110223e-16	-0.754893	-0.570440	-0.323628	True

Рисунок 22 — Проверка найденных решений на допустимость ограничений

3.15. Итоговое сравнение эффективности методов

На заключительном этапе было выполнено итоговое сравнение эффективности реализованных методов. Для этого были определены методы, показавшие лучшие результаты по количеству итераций, времени выполнения и значению целевой функции.

По количеству итераций наилучший результат показал метод модифицированных функций Лагранжа, которому потребовалось 5 итераций. По времени выполнения наименьшее значение также показал метод модифицированных функций Лагранжа. Минимальное значение целевой функции было получено методом внешних штрафных функций и составило приблизительно $f(x^*) \approx 100.4897$.

При этом все методы нашли допустимые решения. Приближенная точка условного минимума имеет вид:

$$x^* \approx (0.7549, 0.5704, 0.3236).$$

Таким образом, реализованные методы позволили найти условный минимум тестовой функции Розенброка при заданных ограничениях и сравнить эффективность различных подходов к решению задачи условной оптимизации.

```
... Best method by number of iterations: Modified Lagrangian method
    Minimum iterations: 5

    Best method by execution time: Modified Lagrangian method
    Minimum execution time: 0.060497

    Best method by function value: External penalty method
    Minimum function value: 100.48972

    Approximate constrained minimum point:
    x* = (0.754892, 0.570441, 0.323629)

    All methods found feasible solutions: True
```

Рисунок 23 — Итоговое сравнение эффективности методов

4) Заключение

В ходе лабораторной работы были изучены и программно реализованы методы поиска условного экстремума. В качестве тестовой задачи рассматривалась минимизация функции Розенброка при наличии ограничений, заданных вариантом 2.

В работе были реализованы следующие методы условной оптимизации: метод внешних штрафных функций, метод внутренних барьерных функций, комбинированный метод штрафных и барьерных функций, метод модифицированных функций Лагранжа и метод проекции градиента. Для каждого метода было найдено приближенное решение, вычислено значение целевой функции, количество итераций, время выполнения, норма градиента и величина нарушения ограничений.

Полученные результаты показали, что все реализованные методы нашли допустимые решения, удовлетворяющие ограничениям задачи. Значения целевой функции оказались близкими и составили приблизительно $f(x^*) \approx 100.49$. Это подтверждает корректность реализации алгоритмов и согласованность полученных результатов.

Наиболее быстрым по количеству итераций оказался метод модифицированных функций Лагранжа. Метод модифицированных функций Лагранжа показал минимальное время выполнения. Метод внешних штрафных функций позволил получить наименьшее значение целевой функции. При этом все методы обеспечили выполнение ограничений с учетом заданной численной точности.

Таким образом, цель лабораторной работы была достигнута: были изучены методы условной оптимизации, выполнена их программная реализация, найден условный минимум тестовой функции Розенброка и проведено сравнение эффективности реализованных алгоритмов.